DataRush City Bike

Predicts how many classic and electric Citi Bikes will be around NYC stations at a given date and 15-minute slot.
A negative binomial demand model (v3, `demanda_net_v3.onnx`) forecasts departures and arrivals per station, with calibrated 80% ranges.
A Next.js dashboard serves it on a map, see docs/dashboard.md.

This README is the guide for evaluators.
It reproduces the data extraction, the evaluation and the training of the final model, from the raw Citi Bike files to the ONNX file the dashboard serves.

Reproduction at a glance

Step	Command	Time	Output
1. Install	`pip install -r notebooks/requirements.txt`		Python environment
2. Download data	see below		`data/raw/*.zip`
3. Clean and extract	`LIMPIEZA_citibike.ipynb`	3 min, 6 GB RAM	`notebooks/data_procesada/`
4. Evaluation protocol	`EVALUACION_protocolo.ipynb`	3 min, 3 GB RAM	Baseline tables in `data/results/`
5. Train the final model	`MODELO_v3.ipynb`	90 min, 4 GB RAM, 8 CPU cores	`notebooks/modelos/demanda_net_v3.onnx`
6. Check the results	compare with `data/results/`		Metrics of docs/model.md

7. Serve it	`cd ui && yarn dev`		Dashboard on http://localhost:3000

No GPU is needed.
Each notebook states what it reads and writes in its first cell.

1. Environment

Requirements: Python 3, about 10 GB of free disk space (the extracted trip files take 8.5 GB), and network access for the weather download.
For the dashboard also Node.js and Yarn.

```
cd notebooks
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

2. Download the data

The model uses 12 consecutive months, Oct 2025 to Sep 2026.
Download the monthly zips from the Citi Bike trip data archive into `data/raw/`:

```
cd data/raw
for m in 202510 202511 202512 202601 202602 202603 202604 202605 202606 202607
202608 202609; do
  curl -O "https://s3.amazonaws.com/tripdata/${m}-citibike-tripdata.zip"
done
```

The months must be consecutive, the cleaning notebook stops if one is missing.
Hourly weather comes from the Open-Meteo archive API and is downloaded and cached by the cleaning notebook, there is nothing to fetch by hand.
More sources: Citi Bike system data and the NYC neighborhoods of docs/nta-dataset.md.

3. Clean the trips and build the training table

```
cd notebooks
jupyter nbconvert --to notebook --execute --inplace LIMPIEZA_citibike.ipynb
```

Or open it with `jupyter lab LIMPIEZA_citibike.ipynb` and run all cells.
It extracts the zips, cleans each trip file, counts departures and arrivals per station, 15-minute slot and bike type, and adds calendar and weather features.
It writes `notebooks/data_procesada/`.

Expected: 45,384,105 raw trips, 45,226,146 after cleaning, 35,040 slots and 81,158,880 rows in `demanda_15min.parquet`.
Steps, outputs and checks: docs/ml-data.md.

4. Run the evaluation protocol

```
jupyter nbconvert --to notebook --execute --inplace EVALUACION_protocolo.ipynb
```

It fixes the train, validation and test blocks, scores the baselines and writes the tables in `data/results/`.
Every model is scored with the same helpers in `citibike_eval.py`.
Protocol and baselines: docs/evaluation.md.

5. Train the final model

```
jupyter nbconvert --to notebook --execute --inplace --
ExecutePreprocessor.timeout=-1 MODELO_v3.ipynb
```

The notebook does the following, see docs/model.md:

1. Compares the model variants on the validation block.
2. Scores five weekly rolling-origin folds.
3. Fits the shared shocks that calibrate the intervals.
4. Fits the final model on all days before Sep 17 (`refit`) and scores it on the test block, Sep 17 to 30.
5. Exports `notebooks/modelos/demanda_net_v3.onnx`, `demanda_net_v3.pt`, `estaciones_onnx_v3.json` and `station_ids_v3.json`.

Fitted models and predictions are cached in `tmp/v3/`, so a rerun takes about 2 minutes.
Delete that folder to retrain everything, which takes about 90 minutes on 8 CPU cores.
Training uses fixed seeds, but small differences in the last digits can appear between machines and library versions.

6. Check the results

The final model (`v3, refit`) on the test block must be close to this table (mean of total departures and total arrivals):

model	deviance_15min	mae_15min	wape_hour	wape_day
v3, refit (ships)	0.7910	0.5060	0.4678	0.2086
v2 network (previous version)	0.8302	0.5163	0.4974	0.2555

Baseline to beat	0.9111	0.5658	0.5674	0.3339

The notebooks write the full tables to `data/results/`, for example `point_metrics_v3.csv`, `v3_test_summary.csv` and `intervals_v3_test.csv`.
The files committed there are the ones the submitted results come from, so a diff shows any change.

7. Serve the model in the dashboard

```
cp notebooks/modelos/demanda_net_v3.onnx ui/models/
cp notebooks/modelos/estaciones_onnx_v3.json ui/models/
cd ui
yarn install
yarn check:reference
yarn dev
```

`yarn check:reference` runs the TypeScript port of the model code against `data/results/ui_reference_v3.json`, which `notebooks/ui_reference.py` computes in Python.
Open http://localhost:3000.
The `ui/models/` folder in the repository already has the JSON files, and `demanda_net_v3.onnx` is gitignored there.

Optional: rebuild the map data

The dashboard map data in `ui/src/data/` is committed.
To rebuild it from the raw trips (standard library only):

```
python3 scripts/extract_stations.py
mkdir -p tmp
curl -sL "https://data.cityofnewyork.us/resource/9nt8-h7nd.geojson?\$limit=1000"
-o tmp/nta.geojson
python3 scripts/assign_regions.py
```

See docs/stations.md and docs/regions.md.
This does not affect the model.

Repository layout

Path	Purpose
`notebooks/`	Cleaning, evaluation, training and findings notebooks, with their Python modules

`notebooks/modelos/`	Trained models and their station files
`data/raw/`	Raw Citi Bike trip files (gitignored)
`data/results/`	Evaluation tables, see docs/evaluation.md
`scripts/`	Station and region preparation for the map
`ui/`	Next.js dashboard and API, see docs/api.md
`slides/`	Slidev pitch deck, in Spanish
`docs/`	Notes on each process, see docs/README.md

More

- docs/findings.md: five findings on what drives demand, from `notebooks/HALLAZGOS_demanda.ipynb`.
- docs/inference.md: how the API runs the ONNX model.
- docs/dashboard.md: the dashboard features.